Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

Dhanush Sai Suprapadha	**PES2UG24CS154**
G S Dheekshith	**PES2UG24CS166**

under the guidance of

Prof. Neha Sharma

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

---

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

Dhanush Sai Suprapadha	**PES2UG24CS154**
G S Dheekshith	**PES2UG24CS166**

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature

Prof. Neha Sharma

Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Operating System** has been carried out by us under the guidance of **Prof. Neha Sharma, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Jan – May 2026.

Dhanush Sai Suprapadha	**PES2UG24CS154**
G S Dheekshith	**PES2UG24CS166**

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Neha Sharma, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project develops a lightweight Linux container runtime in C with a parent supervisor and a kernel-space memory monitor. The runtime can launch and manage multiple isolated containers using Linux namespaces and separate root filesystems, while providing CLI commands to start, inspect, log, and stop containers. A bounded-buffer logging system safely captures container output through synchronized producer–consumer threads.

Alongside the user-space runtime, a Linux Kernel Module monitors container memory usage using registered host PIDs and enforces configurable soft and hard RSS limits through `ioctl` communication. The project also uses controlled workloads to study Linux scheduling behavior under different priorities. Overall, the system acts as a small Docker-like runtime that demonstrates process isolation, IPC, synchronization, kernel memory enforcement, and scheduling concepts in a practical way.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	1
2.	PROBLEM DEFINITION	2
3.	METHODOLOGY	3
4.	CODE	4
5.	RESULT AND DISCUSSION	18
6.	OUTPUTS	20
7.	GITHUB LINK	21
REFERENCES/BIBLIOGRAPHY		22
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		23

INTRODUCTION

Containers are a lightweight way to run isolated applications on the same operating system kernel without the overhead of full virtual machines. They are widely used because they start fast, use fewer resources, and provide good process and filesystem isolation.

In this project, we built a **mini container runtime from scratch** to understand how containers work at the operating system level. The userspace part (engine.c) was used to create and manage multiple containers, track their states, handle commands like start, stop, logs, and ps, and demonstrate process scheduling behavior.

Along with this, we developed a **Linux kernel module (**monitor.c**)** to monitor the memory usage of container processes. The module periodically checks memory consumption, gives a **soft warning** when usage crosses a limit, and performs a **hard kill** if the process exceeds the maximum allowed threshold.

Overall, this project helped us practically explore important OS concepts such as **process isolation, IPC, kernel-user interaction, memory monitoring, scheduling, and lifecycle management**, while implementing a simplified version of a real container engine.

PROBLEM DEFINITION

Modern applications often need isolated and lightweight execution environments that can run multiple workloads efficiently on the same system. Traditional virtual machines provide isolation but consume significant resources and have higher startup overhead.

The problem addressed in this project is to design a **lightweight container runtime system** that can create and manage multiple isolated processes while also monitoring their resource usage. The system should support container lifecycle operations such as start, stop, process listing, and logging, while maintaining metadata for each running container.

In addition, the project aims to solve the challenge of **resource misuse by container processes**, especially excessive memory consumption. For this, a kernel-level monitoring mechanism is required to track process memory usage, generate warnings when soft limits are crossed, and terminate containers that exceed hard memory limits.

The goal is to combine **userspace container management with kernel-level resource enforcement** to build a simplified but practical container supervision system.

METHODOLOGY

The project was implemented in two main parts: a **userspace container engine** and a **kernel memory monitor**.

First, the userspace engine was developed to create isolated container processes using Linux process control mechanisms. It supports commands such as starting, stopping, listing, and logging containers. Each container's metadata, including PID, state, memory limits, and runtime duration, is maintained by the supervisor process.

Next, a Linux kernel module was built to monitor the memory usage of registered container processes. The module periodically checks the RSS memory of each process using a timer-based polling mechanism. If memory usage crosses the soft limit, a warning is generated, and if it exceeds the hard limit, the container process is terminated.

To validate the implementation, different test workloads such as `memory_hog`, `cpu_hog`, and `io_pulse` were executed inside containers. Screenshots of container creation, metadata tracking, log streaming, memory limit enforcement, scheduling behavior, and cleanup were collected as proof of successful execution.

CODE IMPLEMENTATION

The project implementation is divided into two major modules: **the userspace container engine (engine.c)** and **the kernel memory monitor (monitor.c)**.

In **engine.c**, the container runtime was implemented using Linux system calls such as process creation, signal handling, and IPC through UNIX domain sockets. A supervisor process was created to manage multiple containers at the same time. It handles commands like **start**, **stop**, **ps**, and **logs**, allowing the user to control the full container lifecycle. For every container, details such as container ID, PID, execution state, configured memory limits, and runtime duration are stored and updated dynamically. Logging support was also implemented using a bounded-buffer producer–consumer model to stream process outputs.

In **monitor.c**, a Linux kernel module was developed to enforce memory limits on container processes. When a container starts, its PID and memory thresholds are registered with the module through ioctl communication. A timer-based polling mechanism periodically checks the RSS memory usage of each registered process. If the usage crosses the **soft limit**, a warning is printed to the kernel log. If it exceeds the **hard limit**, the module sends a termination signal to the process and unregisters it from the monitoring list.

To verify the correctness of the implementation, custom workloads like **memory_hog**, **cpu_hog**, and **io_pulse** were executed inside containers to test memory enforcement, scheduling behavior, and log streaming respectively.

RESULTS & DISCUSSION

The implemented mini container runtime was successfully tested using multiple workloads, and all major functionalities worked as expected.

The supervisor process was able to **launch and manage multiple containers simultaneously**, while correctly maintaining metadata such as container ID, PID, execution state, memory limits, and runtime duration. The ps command verified that container states were updated dynamically during execution.

The logging mechanism was validated using the io_pulse workload, where repeated output from the container process was successfully streamed and stored. This confirmed the correct working of the **bounded-buffer producer–consumer logging pipeline**.

Memory monitoring results showed that the kernel module successfully detected both **soft-limit and hard-limit violations**. When the memory_hog workload exceeded the soft threshold, warning messages were generated in the kernel log. On crossing the hard limit, the process was terminated automatically, and the container state changed to **killed**, confirming successful kernel-level enforcement.

The scheduling experiment using cpu_hog demonstrated that process priority affected CPU usage as expected. Containers with better nice values received more CPU time, validating scheduler behavior inside the runtime.

During teardown, completed and terminated containers were correctly shown as **exited** or **killed**, and the supervisor cleanup process successfully removed runtime resources. Overall, the results confirmed that the project achieved reliable **container lifecycle management, log handling, memory enforcement, scheduling validation, and cleanup**.

OUTPUT & Screenshots

1) Multi-Container Supervision

This screenshot shows the supervisor daemon successfully running and managing multiple containers under a single control process. The `alpha` and `beta` containers are both active at the same time, confirming that the runtime supports multi-container supervision

```
dheekshith@asus-a15:~/OS-Jackfruit$ cd boilerplate
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo insmod monitor.ko
sudo ./engine supervisor ../rootfs-base
[supervisor] Ready. base-rootfs=../rootfs-base socket=/tmp/mini_runtime.sock
```

```
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ cd ~/OS-Jackfruit/boilerplate
sudo ./engine start alpha ../rootfs-alpha /bin/sh --soft-mib 48 --hard-mib 80
[sudo] password for dheekshith:
started 'alpha' pid=7188
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo ./engine start beta ../rootfs-beta /bin/sh --soft-mib 64 --hard-mib 96
started 'beta' pid=7361
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
ID              PID     STATE     SOFT(MiB)  HARD(MiB)  STARTED
beta            7361    running   64         96         00:07:30
alpha           7188    running   48         80         00:03:16

dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$
```

the successful insertion of the monitor kernel module followed by the startup of the supervisor process, which initializes the base root filesystem and creates the runtime socket for communication. The second screenshot demonstrates the launch of two containers, **alpha** and **beta**, and the ps output confirms that both containers are running simultaneously with their respective PIDs, memory limits, and execution states.

2) Metadata tracking

engine ps output showing tracked container IDs, states, and memory policy status.

```
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
ID              PID     STATE     SOFT(MiB)  HARD(MiB)  STARTED
beta            7361    running   64         96         00:07:30
alpha           7188    running   48         80         00:03:16
```

Displays container metadata including IDs, current states, and applied memory limits.

3)Bounded-buffer Logging

```
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo ./engine start gamma ../rootfs-beta /io_pulse
sleep 3
sudo ./engine logs gamma
started 'gamma' pid=7452
io_pulse wrote iteration=1
io_pulse wrote iteration=2
io_pulse wrote iteration=3
io_pulse wrote iteration=4
io_pulse wrote iteration=5
io_pulse wrote iteration=6
io_pulse wrote iteration=7
io_pulse wrote iteration=8
io_pulse wrote iteration=9
io_pulse wrote iteration=10
io_pulse wrote iteration=11
io_pulse wrote iteration=12
io_pulse wrote iteration=13
io_pulse wrote iteration=14
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo ./engine ps
ID              PID     STATE     SOFT(MiB)  HARD(MiB)  STARTED
gamma           7452    exited    40         64         00:10:14
beta            7361    running   64         96         00:07:30
alpha           7188    running   48         80
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$
```

The screenshot shows the gamma container running the io_pulse workload, where multiple iteration messages are continuously produced and successfully streamed through the logging mechanism. The final ps output also confirms correct state tracking by showing gamma as **exited** after completing execution, while other containers continue to remain active.

4) CLI and IPC

The screenshots show successful command-line interaction with the runtime using operations such as start, stop, logs, and ps. These commands are transmitted from the CLI client to the supervisor through the UNIX domain socket, confirming correct inter-process communication and supervisor response handling.

SSC taken post experiment; showing the supervisor run and exit later after SSC8:

```
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo insmod monitor.ko
sudo ./engine supervisor ../rootfs-base
[sudo] password for dheekshith:
[supervisor] Ready. base-rootfs=../rootfs-base socket=/tmp/mini_runtime.sock
[supervisor] Container iota (pid 8923) -> killed
[supervisor] Container kappa (pid 8985) -> killed
[supervisor] Container mu (pid 9039) -> exited
[supervisor] Container lambda (pid 9032) -> exited
[supervisor] Shutdown signal received.
[supervisor] Shutting down containers...
[logger] Consumer thread done.
[supervisor] Clean exit.
```

5) Soft-Limit Warning

This screenshot captures the kernel module detecting that a container's memory usage crossed the configured soft threshold while running the memory_hog workload. A SOFT LIMIT warning is generated in the kernel log, proving that periodic RSS monitoring and threshold comparison are functioning correctly.

```
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo ./engine start iota ../rootfs-alpha /memory_hog --soft-mib 20 --hard-mib 200
sleep 5
sudo dmesg | grep container_monitor | tail -n 10
started 'iota' pid=8923
[ 2773.376160] [container_monitor] Registering container=eta pid=7763 soft=41943040 hard=67108864
[ 2773.377074] [container_monitor] Unregister request container=eta pid=7763
[ 2783.070680] [container_monitor] Unregister request container=zeta pid=7756
[ 3087.662432] [container_monitor] Process gone: container=alpha pid=7188 - removing
[ 3087.662470] [container_monitor] Process gone: container=beta pid=7361 - removing
[ 3132.894207] [container_monitor] Module unloaded.
[ 5166.577393] [container_monitor] Module loaded. Device: /dev/container_monitor
[ 5186.737195] [container_monitor] Registering container=theta pid=8883 soft=33554432 hard=67108864
[ 5261.738210] [container_monitor] Registering container=iota pid=8923 soft=20971520 hard=209715200
[ 5263.854784] [container_monitor] SOFT LIMIT container=iota pid=8923 rss=25821184 limit=20971520
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$
```

6. Hard-Limit Enforcement

The screenshot shows the container exceeding the configured hard memory threshold, after which the kernel module automatically terminates the process. The ps output confirms the state transition to **killed**, validating successful hard-limit enforcement and userspace state synchronization.

```
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo ./engine start kappa ../rootfs-alpha /memory_hog --soft-mib 20 --hard-mib 30
sleep 5
sudo dmesg | grep container_monitor | tail -n 10
sudo ./engine ps
started 'kappa' pid=8985
[ 5261.738210] [container_monitor] Registering container=iota pid=8923 soft=20971520 hard=209715200
[ 5263.854784] [container_monitor] SOFT LIMIT container=iota pid=8923 rss=25821184 limit=20971520
[ 5286.382453] [container_monitor] HARD LIMIT container=iota pid=8923 rss=210239488 limit=209715200
[ 5286.397352] [container_monitor] Unregister request container=iota pid=8923
[ 5286.397360] [container_monitor] Unregister: entry not found container=iota pid=8923
[ 5397.573353] [container_monitor] Registering container=kappa pid=8985 soft=20971520 hard=31457280
[ 5400.046462] [container_monitor] SOFT LIMIT container=kappa pid=8985 rss=25690112 limit=20971520
[ 5401.070463] [container_monitor] HARD LIMIT container=kappa pid=8985 rss=34078720 limit=31457280
[ 5401.073920] [container_monitor] Unregister request container=kappa pid=8985
[ 5401.073925] [container_monitor] Unregister: entry not found container=kappa pid=8985
ID              PID      STATE      SOFT(MiB)  HARD(MiB)  STARTED
kappa           8985     killed     20         30         01:01:17
iota            8923     killed     20         200        00:59:01
theta           8883     running    32         64
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$
```

7)Scheduling Experiment

```
cpu_hog: running for 30 seconds
cpu_hog: tick 100000000 (elapsed 0s)
cpu_hog: tick 200000000 (elapsed 1s)
cpu_hog: tick 300000000 (elapsed 1s)
cpu_hog: tick 400000000 (elapsed 1s)
cpu_hog: tick 500000000 (elapsed 2s)
cpu_hog: tick 600000000 (elapsed 2s)
cpu_hog: tick 700000000 (elapsed 2s)
cpu_hog: tick 800000000 (elapsed 2s)
cpu_hog: tick 900000000 (elapsed 3s)
cpu_hog: tick 1000000000 (elapsed 3s)
cpu_hog: tick 1100000000 (elapsed 3s)
cpu_hog: tick 1200000000 (elapsed 4s)
cpu_hog: tick 1300000000 (elapsed 4s)
cpu_hog: tick 1400000000 (elapsed 4s)
cpu_hog: tick 1500000000 (elapsed 5s)
cpu_hog: tick 1600000000 (elapsed 5s)
cpu_hog: tick 1700000000 (elapsed 5s)
cpu_hog: tick 1800000000 (elapsed 7s)
cpu_hog: tick 1900000000 (elapsed 10s)
cpu_hog: tick 2000000000 (elapsed 11s)
cpu_hog: tick 2100000000 (elapsed 11s)
cpu_hog: tick 2200000000 (elapsed 12s)
cpu_hog: tick 2300000000 (elapsed 12s)
cpu_hog: tick 2400000000 (elapsed 12s)
cpu_hog: tick 2500000000 (elapsed 13s)
cpu_hog: tick 2600000000 (elapsed 13s)
cpu_hog: tick 2700000000 (elapsed 13s)
cpu_hog: tick 2800000000 (elapsed 14s)
cpu_hog: tick 2900000000 (elapsed 14s)
```

This screenshot demonstrates the execution of CPU-intensive workloads with different niceness values to observe Linux scheduler behavior. The process with higher priority receives significantly higher CPU utilization, confirming that scheduling priority directly affects runtime CPU allocation.

8. Clean Teardown

The final screenshot shows the stopping of active containers, verification of exited and killed states, and cleanup of runtime resources. It also includes the successful termination of supervisor processes and kernel module unload sequence, confirming proper teardown and absence of residual runtime activity.

```
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$ sudo ./engine stop theta
sudo ./engine stop iota
sudo ./engine stop kappa
sudo ./engine stop lambda
sudo ./engine stop mu
sudo ./engine ps
ps aux | grep defunct
sudo pkill -f "./engine supervisor"
sleep 1
sudo rmmod monitor
sudo dmesg | grep container_monitor | tail -n 5
SIGTERM -> 'theta' (pid 8883)
container 'iota' not running
container 'kappa' not running
container 'lambda' not running
container 'mu' not running
ID               PID      STATE      SOFT(MiB)  HARD(MiB)  STARTED
mu               9039     exited     40         64         01:02:42
lambda           9032     exited     40         64         01:02:42
kappa            8985     killed     20         30
dheeksh+    4758  0.0  0.0      0     0 ?        Z    Apr15   0:00 [sd_espeak-ng-mb] <defunct>
dheeksh+    9160  0.0  0.0   9212  2652 pts/2    S+   01:05   0:00 grep --color=auto defunct
[ 5482.570412] [container_monitor] Registering container=mu pid=9039 soft=41943040 hard=67108864
[ 5482.571228] [container_monitor] Unregister request container=mu pid=9039
[ 5492.069727] [container_monitor] Unregister request container=lambda pid=9032
[ 5644.782757] [container_monitor] Process gone: container=theta pid=8883 - removing
[ 5644.835273] [container_monitor] Module unloaded.
dheekshith@asus-a15:~/OS-Jackfruit/boilerplate$
```

GITHUB Link: [OS-Jack repo]

REFERENCES/BIBLIOGRAPHY: [OS-Jackfruit Ref]